

РЕАЛИЗАЦИЯ КЛИЕНТСКОГО ПРИЛОЖЕНИЯ ДЛЯ ВЫЯВЛЕНИЯ АТАК ТИПА «ЧЕЛОВЕК ПОСЕРЕДИНЕ» В КАЧЕСТВЕ СИСТЕМНОЙ СЛУЖБЫ WINDOWS

Степанов П. П.¹, Никонова Г. В.²

DOI: 10.21681/2311-3456-2026-1-128-138

Цель исследования заключается в разработке клиент-серверного приложения для выявления и оперативного реагирования на атаки типа «человек посередине». Клиентское приложение собирает и отправляет данные. Серверное приложение предназначено для тонкой настройки политики безопасности для разных узлов сети.

Методом проведения исследования является разработка сетевой утилиты средствами объектно-ориентированного программирования на языке C#, с реализацией функционала программного обеспечения на уровне классов и методов. Предложен метод разработки функционала на максимально абстрактном уровне – уровне контрактов или интерфейсов (interface), с описанием сигнатуры методов, которые обязан реализовывать класс.

В результате исследования была разработана сетевая утилита, реализованная как системная служба Windows, которая собирает текущую информацию узла сети. В статье описано клиентское приложение этой утилиты. Данная утилита собирает такие параметры как: адрес DHCP сервера; адреса DNS серверов; адрес шлюза; IP адрес текущей машины; имя текущей машины; ARP таблицу; таблицу маршрутизации. Параметры сети передаются для дальнейшего анализа и реагирования на инциденты информационной безопасности.

Практическая ценность – приведена практическая разработка клиентского приложения утилиты. Описана реализация системной службы Windows на языке C#, которая собирает и передает данные для выявления инцидентов, что в свою очередь позволит выявлять и анализировать состояние всех узлов сети (например, для компьютеров одной организации) на предмет атак «человек посередине», таких как: MAC spoofing; ARP spoofing; Evil Twin; DHCP spoofing; ICMP redirect; DNS spoofing; конфигурирование роутера в результате взлома. Итогом работы приложения можно видеть, что системная служба Windows пишет логи о своей работе, в том числе, отображаемые в программе Event Viewer.

Ключевые слова: Windows service, C#, утилита, ARP протокол, DHCP сервер, DNS сервер, ARP-spoofing, перехват трафика, MITM Defense, Evil Twin, ICMP redirect, DHCP spoofing, DNS spoofing, MAC spoofing.

Введение

В настоящее время распределенные технологии и реализующее их программное обеспечение приобретают все большее значение для повышения надежности пересылки и сохранения конфиденциальности пересылаемых сообщений. Современные реалии требуют постоянного повышения безопасности и эффективности этих систем и диктуют задачи, которые невозможно решить без использования научного подхода. В связи с тем, что современные линии связи могут иметь огромную протяженность, сейчас они тянутся на сотни тысяч километров, в результате можно реализовать физическое подключение к каналу, воздействовать на каналы, нарушая функционирование системы [1, 2].

В целом можно сказать, что проблема повышения эффективности и надежности передачи данных в настоящее время очень актуальна и важность проблемы возрастает прямо пропорционально развитию сетевых технологий [3,4]. Это в первую очередь необходимость решать задачи, связанные с уязвимостью

компьютерных сетей, таких как получение несанкционированного доступа к системе как на логическом и физическом уровне, так и в пределах локальной сети или за пределами локальной сети.

Теоретический анализ

Серьезной угрозой при передаче данных являются атаки на компьютерные сети.

В целом алгоритм сетевой атаки можно разделить на следующие этапы:

- 1) Анализ структуры сети, топологии сети, определение открытых портов, версии программного обеспечения и операционной системы.
- 2) Определение возможности проведения сетевых атак на основе обнаруженных уязвимостей и наличия определенных сервисов на атакуемом хосте.
- 3) На основе обнаруженных уязвимостей, выполнение самой атаки.
- 4) Анализ полученных результатов.
- 5) Соккрытие информации об атаке (очистка журналов, закрытие портов).

1 Степанов Петр Петрович, старший преподаватель кафедры Автоматизированные системы обработки информации и управления федерального государственного автономного учреждения высшего образования «Омский государственный технический университет» (ОмГТУ), г. Омск, Россия, E-mail: omsk.petr@gmail.com

2 Никонова Галина Владимировна, кандидат технических наук, доцент, доцент кафедры Автоматизированные системы обработки информации и управления федерального государственного автономного учреждения высшего образования «Омский государственный технический университет» (ОмГТУ), г. Омск, Россия, E-mail: nikonova@omgtu.ru

Если атака будет успешной, злоумышленник нанесет ущерб, похитив данные или выведя из строя атакованный хост [5, 6].

Класс атак на протоколы сетевого обмена с использованием логического доступа может быть реализован путем получения доступа к персональным компьютерам и серверам локальной сети, а также к телекоммуникационному оборудованию. Помимо такого вида есть сетевые атаки, направленные на информационный обмен данными, поскольку информационные протоколы могут иметь ряд недостатков, а также из-за неправильной конфигурации аппаратного и программного обеспечения злоумышленник может получить несанкционированный доступ, а затем осуществить сетевую атаку [7, 8]. После успешного получения доступа, необходимого для проведения сетевой атаки, злоумышленник имеет возможность для перехвата данных и служебной информации, передаваемых по сетям, которые впоследствии могут быть использованы для их модификации, замены исходных данных ложными, перенаправления пакетов [8,9] и т.д. Более того, можно манипулировать не только пользовательскими данными, но и служебной информацией, передаваемой сетевыми узлами, например, таблицами маршрутизации ARP-таблицами [10–12].

Одним из методов предотвращения сетевых атак является использование протоколов, поддерживающих

шифрование данных, таких как SSL [13, 14]. В качестве противодействия атакам на протоколы сетевого обмена применяются логические способы защиты информации, это: проверка подлинности обеих сторон; шифрование передаваемой информации как симметричными, так и асимметричными криптографическими системами; проверка целостности принимаемых данных; защита от удаления и изменения данных [8, 9].

Другим способом защиты данных, передаваемых по открытым каналам связи, является создание защищенных виртуальных сетей (VPN) [14].

Был проведен сравнительный анализ популярных утилит, применяемых для обнаружения атак типа «Man in the middle» (MITM).

Сетевые атаки типа «человек посередине» MITM [15,16] (англ. Man in the middle) являются серьезной проблемой информационной безопасности. Самый простой способ осуществить атаку «человек посередине» — это метод, основанный на изменении IP-адреса на компьютере жертвы. Это изменение может быть внесено локально, с помощью программного обеспечения для удаленного администрирования или с помощью вирусного программного обеспечения. Чтобы изменить IP-адрес в Unix, вам нужно удалить старый адрес шлюза, затем добавить новый адрес, в системах Windows [17], затем вам нужно открыть окно свойств протокола TCP/IP и заполнить

Таблица 1.

Сравнительная таблица утилит для обнаружения MITM-атак

Утилита	Платформы	Методы обнаружения	Преимущества	Недостатки
Wireshark	Windows, Linux, macOS	Анализ пакетов (ARP, DNS, SSL), ручной анализ	Глубокий сетевой анализ, мощный фильтр	Не в реальном времени, требует опыта
Arpwatch	Linux	Мониторинг ARP-таблицы	Прост в настройке, эффективно фиксирует изменения MAC/IP	Только ARP-спуфинг, нет GUI
XArp	Windows, Linux	Анализ ARP и MAC-адресов, профили безопасности	Графический интерфейс, активное предупреждение	Условно бесплатен, возможны ложные срабатывания
Snort	Windows, Linux	IDS: сигнатуры атак (в т.ч. MITM), аномалии	Гибкая настройка правил, активное сообщество	Сложность настройки, требует времени
Suricata	Windows, Linux	IDS/IPS, сигнатуры, эвристика	Многопоточность, высокая производительность	Требуется настройка правил
RogueScanner	Windows	Сканирует локальную сеть на «подозрительные» устройства	Легкость в использовании	Ограничен функционально
MITMDetector	Linux (Python)	Сравнение SSL-сертификатов, DNS spoofing	Прост, кросс-платформенность, полезен для обнаружения прокси и подмены	Ограничен HTTPS-анализом, не обновляется активно
Nmap + NSE	Windows, Linux, Python macOS	Сканирование и скрипты обнаружения MITM	Мощный инструмент в сочетании со скриптами	Требуется ручной работы и знаний
Netcut Defender	Windows	Обнаружение ARP-атак	Простой интерфейс, автоматическое блокирование	Подходит только для Windows, мало гибкости

требуемое поле [3]. По сути это процедура изменения шлюза.

Таким образом, сравнительный анализ методов обнаружения MITM атак и оценке угроз безопасности информации, показал недостаточность значимых механизмов средств защиты.

Решение

Основная идея атаки типа «человек посередине» заключается в том, что трафик не идет напрямую между двумя узлами сети, а проходит через злоумышленника. Злоумышленник может просматривать и модифицировать проходящий через него трафик [18, 19]. Возникла идея в реализации клиент-серверной утилиты для выявления и оперативно реагирования на подобные атаки [20, 21]. Данная утилита должна состоять из клиентского приложения которое будет устанавливаться на все компьютеры из серверного приложения, которая будет анализировать поступающие данные из сервера базы данных, где будут храниться валидные настройки узлов сети, такие как адреса DHCP серверов, адреса DNS серверов, адреса шлюзов.

В статье приведено описание разработанного клиентского приложения под операционную систему Windows (рис. 1). Разработка сетевой утилиты проведено средствами объектно-ориентированного программирования на языке C#, с реализацией программного обеспечения на уровне классов и методов. Функционал разработан на уровне контрактов или интерфейсов (interface), с описанием сигнатуры методов, которые обязаны реализовывать класс. Таким образом предложен метод разработки на максимально абстрактном уровне.

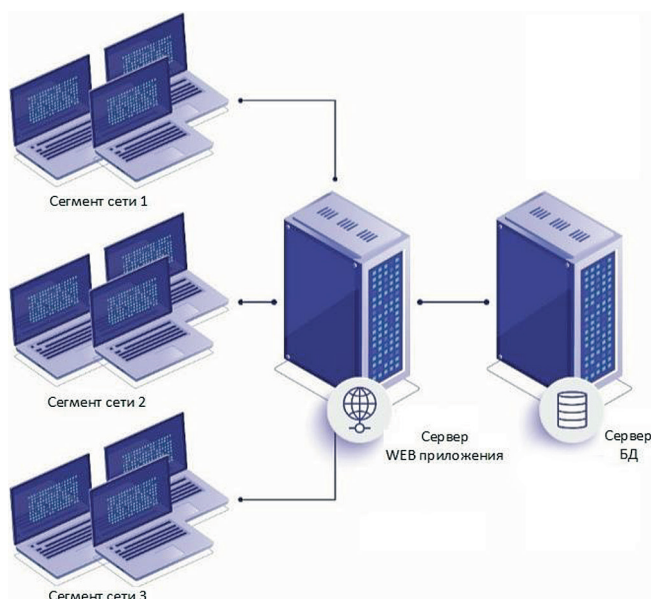


Рис. 1. Схема работы приложения

В отличие от [17, 21], приложение работает как системная служба, собирает и отправляет данные о настройках сети, которые могут быть скомпрометированы при проведении MITM атак; таких как: MAC spoofing; ARP spoofing; Evil Twin; DHCP spoofing; ICMP redirect; DNS spoofing; конфигурирование роутера в результате взлома.

Описание приложения (Solution) NetworkMonitor

На данный момент общая схема решения (Solution) NetworkMonitor состоит из 4-х проектов:

1. NetworkMonitor.Common – сборка содержит в себе базовые вещи, которые используются во всех других сборках (рис. 2):

Constants – константы такие как сообщения об ошибках, сообщения для логирования, значения по умолчанию.

Dto – (Data Transfer Object) – данные для передачи от клиента серверу.

Enums – содержит перечисления.

Exceptions – содержит исключения, которые могут возникать в ходе работы.

ExtensionMethods – содержит методы расширения для базовых классов .net, например, для NetworkInterface.

Interfaces – содержит описание контрактов, которые необходимо реализовать для каждой платформы (Windows, Linux).

Settings – содержит классы с настройками.

2. NetworkMonitor.Implementation – сборка содержит в себе реализации интерфейсов для различных платформ.

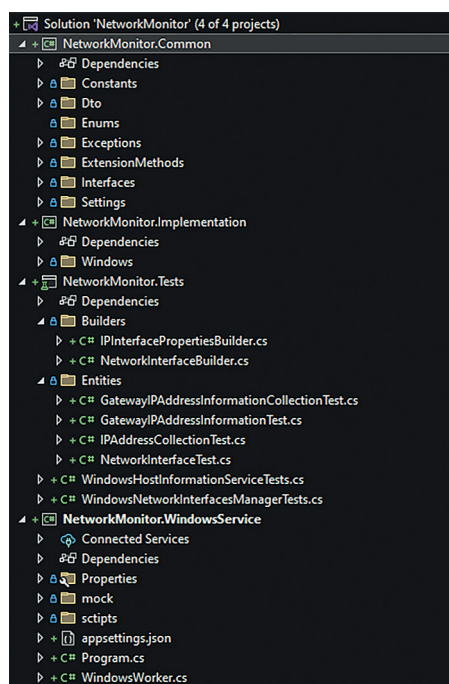


Рис. 2. Схема решения (Solution)

3. *NetworkMonitor.Tests* – сборка содержит в себе Unit тесты для проверки корректной работы различных модулей, папка *Builders* – содержит в себе реализации GoF паттерна Builder для создания сущностей, которые будут использоваться в Unit тестах.

4. *NetworkMonitor.WindowsService* – сборка содержит в себе клиент для системы Windows, которая может быть установлена в систему в качестве системной службы (Windows Services). Также здесь настраивается внедрение нужных зависимостей DI (Dependency injection), необходимых для работы именно под операционной системой Windows [17].

Описание интерфейсов и Dto

Прежде чем приступить к реализации функционала программного обеспечения (ПО) на уровне классов и методов, необходимо продумать функционал на максимально абстрактном уровне – уровне контрактов или интерфейсов (interface), прописав сигнатуры методов, которые обязан реализовывать класс. Это нужно для дальнейшего использования DI (Dependency injection), что уменьшает связность кода и делает его более гибким для изменений и более легким для написания модульных тестов (Unit).

- *IHostInformationService* – интерфейс содержит в себе контракт на получение информации о узле сети, и обязует класс содержать в себе следующие методы:
- *GetDhcp()* – получение IP адреса DHCP сервера. При проведении ряда MITM, таких как DHCP spoofing или Evil Twin он может быть изменен, как приведено в [11, 22].
- *GetGateway()* – получение IP адреса шлюза по умолчанию. Это тот узел сети, через который идет весь трафик по умолчанию. Шлюз может быть изменен в результате взлома и переконфигурирования роутера, атак ICMP redirect или DHCP spoofing, как приведено в [14, 15, 22].
- *GetHostName()* – получение имени машины.
- *GetPv4Address()* – получение IP адреса машины.
- *GetDnsList()* – получение списка IP адресов DNS. Если были изменены адреса DNS серверов, злоумышленник может перенаправить пользователя на поддельный сайт, который выглядит как точная копия легитимного и запросить у пользователя ввести логин и пароль, тут не спасет шифрование посредством SSL, как приведено в [12].
- *GetTracertTable()* – получение таблицы маршрутизации. Нужно для дальнейшего анализа и выявления атак ARP spoofing [13,16] и MAC spoofing. При выполнении этих атак на атакуемой машине будет аномалия, а именно трафик вместо того, чтобы сразу пойти на шлюз, по умолчанию будет проходить еще через один узел сети, что описано в [14,16,23].

- *GetArpTable()* – получение ARP таблицы содержащей коллекцию *Dto Host*. Нужно для дальнейшего анализа и выявления атак ARP spoofing и MAC spoofing. При выполнении этих атак на атакуемой машине будет аномалия, а именно у двух узлов с разными IP адресами будут одинаковые MAC адреса, как это приведено в [14, 16, 23].

На основе анализа приведенных выше уязвимостей авторы предлагают решения для настройки политики безопасности для разных узлов сети в соответствии со схемой решения приведенной на рис. 2.

Программная реализация утилиты (Solution) *NetworkMonitor*

- Фрагмент кода интерфейса *IHostInformationService*:

```
/// <summary> Получение информации об узле
сети. </summary>
public interface IHostInformationService
{
    /// <summary> Получение информации об
    узле сети. </summary>
    HostInformation GetHostInformation();
    /// <summary> Получение IP адреса DHCP
    Сервера. </summary>
    string GetDhcp();
    /// <summary> Получение IP адреса шлюза
    по-умолчанию. </summary>
    string GetGateway();
    /// <summary> Получение имени машины. </
    summary>
    string GetHostName();
    /// <summary> Получение IP адреса маши-
    ны. </summary>
    string GetPv4Address();
    /// <summary> Получение списка IP адре-
    сов DNS. </summary>
    IEnumerable<string> GetDnsList();
    /// <summary> Получение таблицы маршру-
    тизации. </summary>
    IEnumerable<string> GetTracertTable();
}
```

- *GetArpTable()* – получение ARP таблицы для текущего узла сети, содержащий коллекцию *Dto Host* (фрагмент кода):

```
/// <summary> Информация об узле сети. </
summary>
public class Host
{
    /// <summary> IP адрес. </summary>
    public string IPAddress { get; set; }
    /// <summary> MAC адрес. </summary>
    public string MacAddress { get; set; }
}
```

- *GetHostInformation()* – получение информации об узле сети в виде *Dto HostInformation* (фрагмент кода):

```
/// <summary> Информация об узле сети. </
summary>
[Serializable]
public record HostInformation
```

```
{
    /// <summary> IP адрес DHCP Сервера. </summary>
    public string Dhcp { get; set; }
    /// <summary> IP адрес шлюза по-умолчанию. </summary>
    public string Gateway { get; set; }
    /// <summary> Имя машины. </summary>
    public string HostName { get; set; }
    /// <summary> IP адрес машины. </summary>
    public string IPv4Address { get; set; }
    /// <summary> IP адреса DNS серверов. </summary>
    public IEnumerable<string> DnsList {
        get; set; }
    /// <summary> ARP таблица. </summary>
    public IEnumerable<Host> ArpTable {
        get; set; }
    /// <summary> Таблица маршрутизации. </summary>
    public IEnumerable<string> TracertTable {
        get; set; }
}
```

- IWindowsManager – интерфейс содержит в себе контракт для работы со специфическими вещами для ОС Windows, которых нет в стандартных библиотеках .net, таких как работа с протоколами ARP и ICMP. Приведем фрагмент кода IWindowsManager, где GetArpTable() – получение ARP таблицы для текущего узла сети, содержащий коллекцию Dto Host и GetTracertTable() – получение таблицы маршрутизации для текущего узла сети, до шлюза по умолчанию:

```
/// <summary> Работа с Windows. </summary>
public interface IWindowsManager
{
    /// <summary> Получение ARP таблицы. </summary>
    /// <returns> ARP таблица. </returns>
    IEnumerable<Host> GetArpTable();
    /// <summary> Получение таблицы маршрутизации. </summary>
    /// <param name="gateway"> IP адрес шлюза по-умолчанию. </param>
    /// <returns> Таблица маршрутизации. </returns>
    IEnumerable<string> GetTracertTable(string gateway);
}
```

- IHttpClient – интерфейс содержит в себе контракт для отправки, данных на сервер посредством протокола HTTP. Содержит в себе сигнатуру метода SendHostInformation для отправки Dto HostInformation с информацией о текущем узле сети на сервер для дальнейшей валидации. Приведем фрагмент кода интерфейса IHttpClient, реализующий метод SendHostInformation:

```
/// <summary> Отправка данных на сервер по протоколу HTTP. </summary>
public interface IHttpClient
{
    /// <summary> Отправка данных сети текущего узла на сервер. </summary>
    /// <param name="hostInformation"> Информация об узле сети. </param>
    void SendHostInformation(HostInformation hostInformation);
}
```

Описание интерфейсов для Windows

- WindowsHttpClientMock – класс реализующий интерфейс IHttpClient, так как сервер еще не реализован, класс представляет собой заглушку для тестирования работоспособности сервиса. Данный метод сериализует экземпляр класса HostInformation в формат json и сохраняет его в файл с именем текущей даты и времени. Класс WindowsHttpClientMock содержит реализацию метода SendHostInformation (фрагмент кода):

```
/// <summary> Класс заглушка для IHttpClient. </summary>
public class WindowsHttpClientMock : IHttpClient
{
    /// <summary> Сохранение информации об узле сети в json файл. </summary>
    public void SendHostInformation(HostInformation hostInformation)
    {
        if (!Directory.Exists("mock"))
        {
            Directory.CreateDirectory("mock");
        }
        var fileName = $"mock\\{DateTime.Now:yyyy_MM_dd-HH_mm_ss}.json";
        using (var stream = File.CreateText(fileName))
        {
            var serializer = new JsonSerializer();
            serializer.Serialize(stream, hostInformation);
        }
    }
}
```

- WindowsCmdManager – пример реализации интерфейса IWindowsManager с помощью стандартных утилит Windows, а именно arp.exe и tracert.exe. WindowsCmdManager – класс реализующий интерфейс IWindowsManager (фрагмент кода).

```
/// <summary> Работа с утилитами командной строки Windows. </summary>
public class WindowsCmdManager : IWindowsManager
{
    private List<string> _tracertTable;
```

```

private List<Host> _tracertArp;
private int _tracertIteration = 0;
private int _arpIteration = 0;
/// <summary> Получение таблицы маршру-
тизации через cmd. </summary>
/// <param name="gateway"> IP адрес
шлюза по-умолчанию. </param>
/// <returns> Таблицы маршрутизации. </
returns>
public IEnumerable<string>
GetTracertTable(string gateway)
{
    _tracertIteration = 0;
    _tracertTable = new List<string>();
    Process cmdProcess = new Process();
    cmdProcess.StartInfo = CreateProcess
StartInfo("tracert", gateway);
    cmdProcess.OutputDataReceived +=
ParseTracert;
    cmdProcess.EnableRaisingEvents = true;
    cmdProcess.Start();
    cmdProcess.BeginOutputReadLine();
    cmdProcess.WaitForExit();
    return _tracertTable;
}
/// <summary> Получение ARP таблицы че-
рез cmd. </summary>
/// <returns> ARP таблица. </returns>
public IEnumerable<Host> GetArpTable()
{
    _arpIteration = 0;
    _tracertArp = new List<Host>();
    Process cmdProcess = new Process();
    cmdProcess.StartInfo = CreateProcess-
StartInfo("arp", "-a");
    cmdProcess.OutputDataReceived +=
ParseArp;
    cmdProcess.EnableRaisingEvents = true;
    cmdProcess.Start();
    cmdProcess.BeginOutputReadLine();
    cmdProcess.WaitForExit();
    return _tracertArp;
}

```

- В методах GetTracertTable и GetArpTable происходит вызов этих утилит с перенаправлением потока вывода на методы ParseTracert и ParseArp и дальнейшего парсинга результата работы утилит arp.exe и tracert.exe (фрагмент кода):

```

private void ParseTracert(object sender,
DataReceivedEventArgs e)
{
    if (_tracertIteration > 3 &&
!string.IsNullOrEmpty(e.Data))
    {
        var tracertString = e.Data
.Replace("[", "")
.Replace("]", "")
.Split(" ")
.LastOrDefault(i => i != "");
        if (!string.
IsNullOrEmpty(tracertString) && IPAddress.

```

```

TryParse(tracertString, out var address))
    {
        _tracertTable.Add(address.
ToString());
    }
    _tracertIteration++;
}
private void ParseArp(object sender,
DataReceivedEventArgs e)
{
    if (_arpIteration > 2 && !string.
IsNullOrEmpty(e.Data))
    {
        var arpString = e.Data
.Split(" ")
.Where(i => !string.IsNullOrEmpty(i))
.ToList();
        if (arpString.Any() && arpString.
Count() == 3
&& IPAddress.TryParse
(arpString.FirstOrDefault(), out var address)
&& arpString.LastOrDefault()
== "dynamic")
        {
            _tracertArp.Add(new Host()
{
            IPAddress = arpString[0],
            MacAddress = arpString[1]
});
        }
        _arpIteration++;
}

```

- WindowsHostInformationService – класс реализующий интерфейс IHostInformationService. Класс WindowsHostInformationService содержит в себе методы, получающие информацию об узле сети. В классе WindowsHostInformationService есть две приватный переменный доступных только для чтения _ipInterfaceProperties типа IPInterfaceProperties и _manager типа IWindowsManager. В конструкторе с помощью такой техники как DI (Dependency injection) происходит внедрение зависимостей, делается это для большей гибкости и для удобства написания Unit тестов, так как можно указать любую реализацию, реализующую интерфейс. Так же класс WindowsHostInformationService содержит реализацию всех методов интерфейса IHostInformationService. Приведем фрагмент кода реализации класса WindowsHostInformationService:

```

/// <summary> Получение информации об узле
сети Windows. </summary>
public class WindowsHostInformationService
: IHostInformationService
{
    private readonly IPInterfaceProperties

```

```

_ipInterfaceProperties;
private readonly IWindowsManager _
manager;
public WindowsHostInformationService(I
PInterfaceProperties ipInterfaceProperties,
IWindowsManager manager)
{
    _ipInterfaceProperties = ipInter-
faceProperties;
    manager = manager;
}
public HostInformation GetHostInforma-
tion()
{
    return new HostInformation
    {
        Dhcp = GetDhcp(),
        DnsList = GetDnsList(),
        Gateway = GetGateway(),
        HostName = GetHostName(),
        IPv4Address = GetPv4Address(),
        TracertTable = GetTracertTable(),
        ArpTable = GetArpTable()
    };
}
}

```

- Методы GetTracertTable и GetArpTable внутри себя вызывают методы из переменной _manager, где может быть любой экземпляр класса реализующий интерфейс IWindowsManager (фрагмент кода):

```

public string GetDhcp()
{
    return _ipInterfaceProperties.
DhcpServerAddresses.FirstOrDefault()?.
ToString();
}
public string GetGateway()
{
    return _ipInterfaceProperties.
GatewayAddresses.FirstOrDefault().Address.
MapToIPv4().ToString();
}
public string GetHostName()
{
    return Environment.MachineName;
}
public string GetPv4Address()
{
    return _ipInterfaceProperties
.UnicastAddresses
.FirstOrDefault(a => a.IPv4Mask.
Address != 0)
.Address.MapToIPv4().ToString();
}
public IEnumerable<string> GetDnsList()
{
    return _ipInterfaceProperties.
DnsAddresses.Select(i => i.MapToIPv4().
ToString()).ToList();
}
public IEnumerable<string>
GetTracertTable()
{
    return _manager.
GetTracertTable(GetGateway());
}

```

```

}
public IEnumerable<Host> GetArpTable()
{
    return _manager.GetArpTable();
}
}

```

Описание NetworkMonitor.WindowsService

- NetworkMonitor.WindowsService – сборка для создания клиента в виде службы window. Для этого необходимо добавить к проекту следующие Nuget пакеты Microsoft.Extensions.Hosting, Microsoft.Extensions.Hosting, WindowsServices и ссылка на проекты NetworkMonitor.Common и NetworkMonitor.Implementation. Файл Program.cs отвечает за конфигурацию сервиса и получение параметров из файла appsettings.json. Далее производится внедрение зависимостей в IoC контейнер, а также добавление WindowsWorker в котором будет выполняться все логика. Приведен код из файла Program.cs в котором происходит конфигурация сервиса:

```

var configuration = new
ConfigurationBuilder()
.AddEnvironmentVariables()
.AddCommandLine(args)
.AddJsonFile("appsettings.json")
.Build();
IHost host = Host.CreateDefaultBuilder
(args)
.UseWindowsService(config =>
{
    config.ServiceName = "NetworkMonitor.
WindowsService";
})
.ConfigureServices(services =>
{
    // Получение настроек.
    var networkInterfaceSetting =
configuration.GetSection("NetworkInterfaceSet-
ting").Get<NetworkInterfaceSetting>();
    var httpClientSetting = configuration.
GetSection("HttpClientSetting").
Get<HttpClientSetting>();
    var ipProperties = NetworkInterface
.GetAllNetworkInterfaces()
.GetNetworkInterfaceByName(network
InterfaceSetting)
.GetIPProperties();
    // Внедрение зависимостей для
Windows Service.
    services.AddSingleton<IWindows-
Manager, WindowsCmdManager>();
    services.AddSingleton<IInterfacePro-
perties>(ipProperties);
    services.AddSingleton<IHostInformati-
onService, WindowsHostInformationService>();
    services.AddSingleton<IHttpClient,
WindowsHttpClientMock>();
    services.AddSingleton<HttpClientSett-
ing>(httpClientSetting);
    services.AddHostedService<WindowsWor-
ker>();
}

```

```
    })  
    .Build();  
    host.Run();
```

- Метод `ExecuteAsync` в котором происходит получение настроек узла и отправка их на сервер (на время, пока с помощью заглушки пишется в json файл). Затем идет задержка, значение которой берется из файла конфигурации и итерация повторяется. Показан код работы метода `ExecuteAsync` из файла `WindowsWorker.cs`:

```
protected override async Task ExecuteAsync  
(CancellationToken stoppingToken)  
{  
    try  
    {  
        while (!stoppingToken.IsCancellationRequested)  
        {  
            _logger.LogInformation("Получение настроек сети.");  
            var hostInformation = _hostInformationService.GetHostInformation();  
            _logger.LogInformation($"Настройки сети получены. {hostInformation.Log()}");  
            _httpClient.SendHostInformation(hostInformation);  
            _logger.LogInformation($"InfoMessages.HttpClientMessageSent");  
            await Task.Delay(_clientSetting.Delay > 5000  
                ? _clientSetting.Delay : DefaultValues.SendHostInformationDelay,  
                stoppingToken);  
        }  
    }  
    catch (Exception e)  
    {  
        _logger.LogError(e.Message);  
    }  
}
```

- Для создания сервиса Windows необходимо скомпилировать сборку `NetworkMonitor.WindowsService` и пополнить bat скрипт правами администратора указав в нем путь до `NetworkMonitor.WindowsService.exe` «create `NetworkMonitor.WindowsService.bat`» код которого представлен ниже:

```
sc create NetworkMonitor.WindowsService  
binPath= "E:\Source\NetworkMonitor.WindowsService.exe"  
sc description NetworkMonitor.WindowsService "NetworkMonitor.WindowsService"
```

- После чего можно запустить утилиту `services.msc` и убедиться, что создана новая служба, которую можно настроить и запускать или выключать.
- По умолчанию для служб Windows используется папка `C:\Windows\System32`. Можно увидеть, что

`NetworkMonitor.WindowsService` создал папку `mock` (рис. 3) и сохраняет туда значение `Dto HostInformation` сериализуя их в json файл.

- Запустив утилиту `Event Viewer`, можно увидеть, что системная служба пишет логи о своей работе рис. 4.

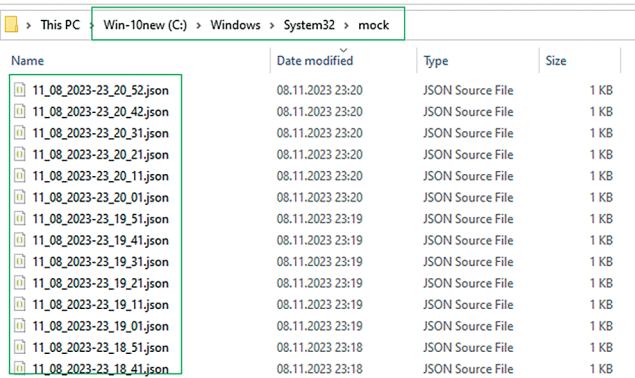


Рис. 3. Папка `mock`

Пример json файла:

```
{  
  "Dhcp": "192.168.1.1",  
  "Gateway": "192.168.1.1",  
  "HostName": "DRUID-HOME-PC",  
  "IPv4Address": "192.168.1.1",  
  "DnsList": "192.168.1.1",  
  "Dhcp": [  
    "192.168.1.1"  
  ],  
  "ArpTable": [  
    {  
      "IPv4Address": "192.168.1.1",  
      "MacAddress": "60-45-cb-90-cf-2c"  
    }  
  ],  
  "TrasertTable": [  
    "192.168.1.1"  
  ]  
}
```

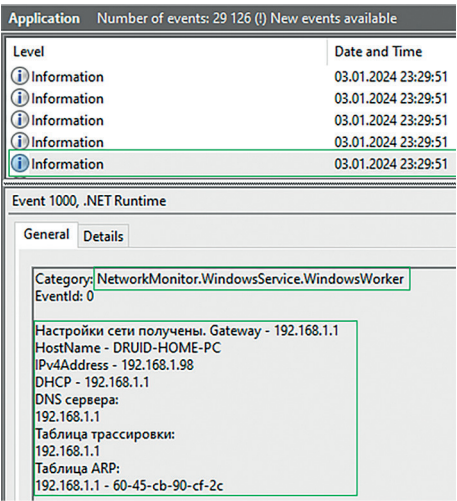


Рис. 4. Логи Windows отображаемые в программе `Event Viewer`

Заключение

Описана разработка сетевой утилиты, работающей как системная служба Windows. Основная идея разработки: написать сетевое клиент-серверное приложение, которое позволяло бы централизованно выявлять атаки MITM («человек посередине») и оперативно реагировать на инциденты. В статье описывается реализация первой части этого приложения, а именно клиента, которое планируется устанавливать на каждый узел сети, для отправки информации

о сетевых настройках, а также ARP таблицы и таблицы трассировки. Описана архитектура клиент-серверного приложения, разработка программного обеспечения, а также показан процесс компиляции и запуска приложения клиента в качестве системной службы Windows. В итоге можно сделать вывод, что разработанная утилита отличается выявлением большого числа MITM атак, имеет клиент-серверную архитектуру и обладает тонкой настройкой правил для каждого узла сети.

Литература

1. Andrew S. Tanenbaum, Todd Austin. Structured computer organization. 6th ed. Pearson Education, Inc., ISBN-13: 978-0-13-291652-3. 2023. 992 p.
2. Олифер В. Г., Олифер Н. А. Компьютерные сети. Принципы, технологии, протоколы. Санкт-Петербург: Питер. 2023. 1008 с. ISBN: 978-5-4461-1426-9.
3. Snaider J. Effective TCP/IP Programming. 44 Tips to Improve Your Network Programs. Addison-Wesley Professional, 2020. 320 p. ISBN-100201615894.
4. Биджиева С. Х., Шебзухова К. В. Сетевые протоколы передачи данных: преимущества и недостатки // Тенденции развития науки и образования. 2022. Т. 86. № 1. С. 43–45. DOI: 10.18411/trnio-06-2022-14.
5. Penetration Testing Active Reconnaissance Phase – Optimized Port Scanning with Nmap Tool / M. Shah, S. Ahmed, K. Saeed, M. Junaid [et al.] // 2019 2nd International Conference on Computing, Mathematics and Engineering Technologies (iCoMET). IEEE, 2019. P. 1–6. DOI: 10.1109/ICOMET.2019.8673520.
6. Muthalagu R., Sanjay S. Evil Twin Attack Mitigation Techniques in 802.11 Networks // International Journal of Advanced Computer Science and Applications. 2021. Vol. 12, no. 6. P. 38–41.
7. Mehrotra T. A review on attack in wireless and computer networking // Asian Journal of Multidimensional Research. 2021. Vol. 10, no. 10. P. 1457–1463.
8. Степанов П. П., Никонова Г. В. Безопасная передача сообщений с разделением данных через почтовые серверы // Вопросы кибербезопасности. 2024. № 2(60). С. 120–129. DOI: 10.21681/2311-3456-2024-2-120-129.
9. Man K., Zhou X., Qian Z. DNS cache poisoning attack: Resurrections with side channels // CCS '21: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security. 2021. P. 3400–3414.
10. GaTeBaSep: game theory-based security protocol against ARP spoofing attacks in software-defined networks / F. Mvuh, V. Kengne Tchendji, C. Tayou Djamegni [et al.] // International Journal of Information Security. 2024. Vol. 23, No. 1. P. 373–387.
11. Attack on the Address Resolution Protocol / P. Stepanov, G. Nikonova, T. Pavlychenko, A. Gil // 2020 International Conference Engineering and Telecommunication (En&T). IEEE, 2020. P. 1–3. DOI: 10.1109/EnT50437.2020.9431297.
12. Zhang Z., Liu Z., Bai J. Network attack detection model based on Linux memory forensics // 2022 14th International Conference on Measuring Technology and Mechatronics Automation (ICMTMA). IEEE, 2022. P. 931–935. DOI: 10.1109/ICMTMA54903.2022.00189.
13. Galal A. A., Ghalwash A. Z., Nasr M. A new approach for detecting and mitigating address resolution protocol (ARP) poisoning // International Journal of Advanced Computer Science and Applications. 2022. Vol. 13, no. 6. P. 377–382. DOI: 10.14569/IJACSA.2022.0130647.
14. The problem of security address resolution protocol / P. P. Stepanov, G. V. Nikonova, T. S. Pavlychenko, A. S. Gil // Journal of Physics: Conference Series. 2021. Vol. 1791. P. 012061. DOI: 10.1088/1742-6596/1791/1/012061.
15. Shah Z., Cosgrove S. Mitigating ARP cache poisoning attack in software-defined networking (SDN): a survey // Electronics. 2019. Vol. 8., No. 10. P. 1095. <https://doi.org/10.3390/electronics8101095>.
16. Особенности работы протокола разрешения адресов в компьютерных сетях / П. П. Степанов, Г. В. Никонова, Т. С. Павлюченко, В. В. Соловьев // Программная инженерия. 2022. Т. 13, № 5. С. 211–218. DOI 10.17587/prin.13.211-218.
17. Долгачев М. В., Костюнин В. А. Комплексный анализ поведения системы WINDOWS для обнаружения киберугроз // Вопросы кибербезопасности. 2025. № 2(66). С. 71–77. DOI: 10.21681/2311-3456-2025-2-71-77.
18. Hijazi S., Obaidat M. S. Address resolution protocol spoofing attacks and security approaches: A survey // Security and Privacy. 2019. Vol. 2, No. 1. P. e49.
19. Shaw S., Choudhury P. A new local area network attack through IP and MAC address spoofing // 2015 international conference on advances in computer engineering and applications. IEEE, 2015. P. 347–350. DOI: 10.1109/ICACEA.2015.7164728.
20. Novokhrestov A., Konev A., Shelupanov A. Model of Threats to Computer Network Software // Symmetry. 2019. Vol. 11, Iss. 12. P. 1506.
21. Maximov R. V., Sokolovsky S. P., Telenga A. P. Model of client-server information system functioning in the conditions of network reconnaissance // CEUR Workshop Proceedings : Selected Papers of the X Anniversary International Scientific and Technical Conference on Secure Information Technologies (BIT 2019). Moscow : CEUR Workshop Proceedings, 2019. Vol. 2603. P. 44–51.
22. DHCP attacking tools: an analysis / M. Aldaoud, D. Al-Abri, A. Al. Maashri, F. Kausar // Journal of Computer Virology and Hacking Techniques. 2021. Vol. 17, no. 2. P. 119–129.
23. Wan K., Coffman J. Game-theoretic modeling of DDoS attacks in cloud computing // Proceedings of the 14th IEEE/ACM International Conference on Utility and Cloud Computing. 2021. P. 1-10. DOI:10.1145/3468737.3494093.

IMPLEMENTATION OF A CLIENT APPLICATION FOR IDENTIFYING THE «MAN-IN-THE-MIDDLE» ATTACKS AS A WINDOWS SYSTEM SERVICE

Stepanov P. P.³, Nikonova G. V.⁴

Keywords: Windows service, C#, Utility, ARP Protocol, DHCP server, DNS server, ARP spoofing, traffic interception, MITM Defense, Evil Twin, ICMP redirect, DHCP spoofing, DNS spoofing, MAC spoofing.

The purpose of this study is to develop a client-server application for identifying and prompt reacting to the attacks of the «man-in-the-middle» type. This application is intended for fine tuning the security policies for different network nodes.

The method for this study is the development of a network utility program by means of object-oriented programming in C#, with the implementation of the software functionality at the level of classes and methods. A method for developing the functionality is proposed at the most abstract level – contracts or interfaces – with a description of the methods signature, which the class should implement.

As a result of this study, a utility program has been developed, which acts as a Windows system service and analyzes the continuous data of a network node. This study describes the client application of this utility program. The utility program collects such parameters as DHCP server address; DNS server addresses; gateway address; IP address of the active computer; the name of the active computer; ARP table and the routing table. These network parameters are transmitted for further analysis and response to information security incidents.

The practical value of this study is the development of the client application of this utility program. The implementation of a Windows system service in C#, which transmits data that helps revealing the attacks, which in turn helps identifying and analyzing the status of all network nodes (for example, computers of the same organization) as to detect a «man-in-the-middle» attack, such as: MAC spoofing; ARP Spoofing; Evil Twin; DHCP Spoofing; ICMP Redirect; DNS Spoofing; configuration of hacked router. The result of running the application could be observed in the Windows logs through Event Viewer application.

References

1. Tanenbaum, Andrew S., Austin, Todd. (2023). Structured computer organization. 6th ed. Pearson Education, Inc., published as Prentice Hall. 776 p.
2. Olifer, V. G., Olifer, N. A. (2023). Computernye seti. Principy, Technologie, Protocols. SPb. Piter. 1008 p. (in Russ.)
3. Snaider, J. (2020). Effective TCP/IP Programming. 44 Tips to Improve Your Network Programs. Pearson Education. 320 p.
4. Bidzhieva, S. H., Shebzuhova, K. V. (2022). Network data transfer protocols: advantages and disadvantages. Trends in the development of science and education, 1(86), 43–45. DOI: 10.18411/trnio-06-2022-14 (in Russ.)
5. Shah, M., Ahmed, S., Saeed, K., Junaid, M., Hamayun, Khan, Ata-ur Rehman. (2019). Penetration Testing Active Reconnaissance Phase – Optimized Port Scanning With Nmap Tool. 2nd International Conference on Computing, Mathematics and Engineering Technologies (iCoMET), IEEE, DOI: 10.1109/ICOMET.2019.8673520.
6. Muthalagu, R., Sanjay, S. (2021). Evil Twin Attack Mitigation Techniques in 802.11 Networks. International Journal of Advanced Computer Science and Applications, 6 (12), P. 38–41.
7. Mehrotra, T. (2021). A review on attack in wireless and computer networking. Asian Journal of Multidimensional Research, 10(10), 1457–1463.
8. Stepanov, P. P., Nikonova, G. V. (2024). Bezopasnaya peredacha soobshcheniy s razdeleniem dannykh cherez pochtovye server. Voprosy kiberbezopasnosti, 2(60), 120–129. DOI: 10.21681/2311-3456-2024-2-120-129. (in Russ.)
9. Man, K., Zhou, X., Qian, Z. (2021). DNS cache poisoning attack: Resurrections with side channels. CCS '21. Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, 3400–3414.
10. Mvah, F., Kengne, V., Tchendji, C. Tayou. (2024). Djamegni et al. GaTeBaSep: game theory-based security protocol against ARP spoofing attacks in software-defined networks. International Journal of Information Security, 1(23), 373–387.
11. Stepanov, P. P., Nikonova, G. V., Pavlychenko, T. S., Gil, A. S. (2020). Attack on the Address Resolution Protocol. 2020 International Conference Engineering and Telecommunication (En&T), IEEE, DOI: 10.1109/EnT50437.2020.9431297.
12. Zhang, Z., Liu, Z., Bai, J. (2022). Network Attack Detection Model Based on Linux Memory Forensics. 2022 14th International Conference on Measuring Technology and Mechatronics Automation, ICMTMA, IEEE, 2022, 931–935. DOI: 10.1109/ICMTMA54903.2022.00189.
13. Galal, A. A., Ghalwash, A. Z., Nasr, M. (2022). A new approach for detecting and mitigating address resolution protocol (ARP) poisoning. International Journal of Advanced Computer Science and Applications, 6(13), P. 377–382. DOI: 10.14569/IJACSA.2022.0130647.
14. Stepanov, P. P., Nikonova, G. V., Pavlychenko, T. S., Gil, A. S. (2021). The problem of security address resolution protocol // Journal of Physics: Conference Series, 1791, P. 012061. DOI: 10.1088/1742-6596/1791/1/012061.
15. Shah, Z., Cosgrove, S. (2019). Mitigating ARP Cache Poisoning Attack in Software-Defined Networking (SDN): A Survey. Electronics, 8(10), 1095. DOI: 10.3390/electronics8101095.
16. Stepanov, P. P., Nikonova, G. V., Pavlychenko, T. S., Solovov, V. V. (2022). Osobennosti raboty protokola razrescheniya adresov v puterneh setyach. Programnaya Ingeneria. 5(13), 211–218. DOI 10.17587/prin. 13. 211–218 (in Russ.)
- 3 Petr P. Stepanov, Senior Lecturer of the Department of Automated Information Processing and Control Systems of the Federal State Autonomous Institution of Higher Education «Omsk State Technical University» (OmSTU), Omsk, Russia, E-mail: omsk.petr@gmail.com
- 4 Galina V. Nikonova, Ph.D. of Technical Sciences, Associate Professor, Associate Professor of the Department of Automated Information Processing and Control Systems of the Federal State Autonomous Institution of Higher Education «Omsk State Technical University» (OmSTU), Omsk, Russia, E-mail: nikonova@omgtu.ru

17. Dolgachev, M. V., Kostynin, V. A. (2025). Kompleksnuy analiz povedeniya sistemy WINDOWS dlya obnaruzheniya kiberugroz. Voprosy kiberbezopasnosti, 2(66), 71–77. DOI: 10.21681/2311-3456-2025-2-71-77.
18. Hijazi, S., Obaidat, M. (2019). Address resolution protocol spoofing attacks and security approaches: A survey. Security and Privacy, 1(2), e49.
19. Shaw, S., Choudhury, P. (2015). A new local area network attack through IP and MAC address spoofing. 2015 international conference on advances in computer engineering and applications. IEEE, 347–350. DOI: 10.1109/ICACEA.2015.7164728.
20. Novokhrestov, A., Konev, A., Shelupanov, A. (2019). Model of Threats to Computer Network Software. Symmetry, 2019, 12(11), 1506.
21. Maximov, R. V., Sokolovsky, S. P., Telenga, A. P. Model of client-server information system functioning in the conditions of network reconnaissance. CEUR Workshop Proceedings : Selected Papers of the X Anniversary International Scientific and Technical Conference on Secure Information Technologies (BIT 2019). Moscow : CEUR Workshop Proceedings, 2603, 44–51.
22. Aldaoud, M. Al-Abri, D., Maashri, A. Al, Kausa F. (2021). DHCP attacking tools: an analysis. Journal of Computer Virology and Hacking Techniques, 2(17), 119–129.
23. Wan, K., Coffman, J. (2021). Game-theoretic modeling of DDoS attacks in cloud computing. Proceedings of the 14th IEEE/ACM International Conference on Utility and Cloud Computing. P. 1-10. <https://doi.org/10.1145/3468737.3494093>.

